ETL Data Transformation Report — Airline Analytics DWH

Projet : Airline Analytics Data Warehouse
Date : 2026-04-24
Outil ETL : SQL Server Integration Services (SSIS) — Visual Studio 2022
Pipeline : 4 CSV → STAGING_DB (raw_ + stg_) → DWH_DB (5 DIM + 2 FACT)

---

1. Architecture du pipeline SSIS

```
┌──────────────────────────────────────────────────────────────────────┐
│  Master.dtsx  (orchestrateur — Execute Package Tasks en séquence)    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ 00_Create_Database.dtsx                                      │   │
│  │  SQL_Reset_Staging → SQL_Create_DWH_DB → SQL_Create_DWH_Tables  │   │
│  │                                        → SQL_Unknown_Members    │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                       │
│  ┌──────────────────────────▼───────────────────────────────────┐   │
│  │ 01_Load_Staging.dtsx                                         │   │
│  │  Prepare → [BulkLoad×4 ‖] → [Transform×4 ‖] → Rebuild → Validate  │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                       │
│  ┌──────────────────────────▼───────────────────────────────────┐   │
│  │ 02_Populate_Dimensions.dtsx                                  │   │
│  │  [DIM_DATE ‖ DIM_LOYALTY_CARD ‖ DIM_GEOGRAPHY ‖ DIM_TRAVEL]    │   │
│  │                      → DIM_CUSTOMER                          │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                       │
│  ┌──────────────────────────▼───────────────────────────────────┐   │
│  │ 03_Populate_Facts.dtsx                                       │   │
│  │  [FACT_PASSENGER_SATISFACTION ‖ FACT_FLIGHT_ACTIVITY]        │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                              │                                       │
│  ┌──────────────────────────▼───────────────────────────────────┐   │
│  │ 04_Finalize.dtsx                                             │   │
│  │  [Index_Fact ‖ Index_Dim] → Update_Stats → Validate_DWH      │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

Connexion : OLE DB → `MEA-JJG4XL3\DEV3` | Provider : `MSOLEDBSQL.1` | Auth : SSPI
Bases de données : `STAGING_DB` (staging) et `DWH_DB` (entrepôt)
Protection : `DontSaveSensitive` (ProtectionLevel=0) — compatible avec toutes les machines
Validation retardée : `DTS:DelayValidation="True"` sur tous les sous-packages (évite la validation design-time contre des objets non encore créés)

---

2. Sources de données

Fichier	Lignes	Encodage	Séparateur	Quote	Remarque
`airline_passenger_satisfaction.csv`	129 880	UTF-8	`,`	`"`	Arrival Delay contient des NULLs
`Customer Flight Activity.csv`	392 936	UTF-8 BOM	`,`	`"`	Loyalty Number en 1ère col (BOM géré par CODEPAGE=65001)
`Customer Loyalty History.csv`	16 737	UTF-8 BOM	`,`	`"`	Cancellation Year/Month vides = membres actifs
`Calendar.csv`	2 557	UTF-8	`,`	`"`	Données propres — 2012-01-01 à 2018-12-31

Paramètres BULK INSERT communs :

```
FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='\n',
CODEPAGE='65001', FIELDQUOTE='"', TABLOCK
```

---

3. Package 00 — `00_Create_Database.dtsx`

Tâches (en séquence)

SQL_Reset_Staging

- Crée le schéma `etl` et la table `etl.run_log` si absents
- Supprime les tables résiduelles (`raw_*`, `stg_*`) d'éventuelles exécutions incomplètes
- Recrée les 4 tables `stg_` avec les **types finaux corrects** (zone typed staging)

Tables stg_ créées :

```
-- stg_airline_satisfaction
ID                INT
Gender            NVARCHAR(50)
Age               INT
CustomerType      NVARCHAR(50)
TypeOfTravel      NVARCHAR(50)
Class             NVARCHAR(50)
FlightDistance    INT
DepartureDelay    INT NULL          -- NULL accepté
ArrivalDelay      INT NULL          -- NULL accepté
TimeConvenience   INT               -- scores 1-5 (0 = non applicable)
OnlineBooking     INT
CheckinService    INT
OnlineBoarding    INT
GateLocation      INT
OnboardService    INT
SeatComfort       INT
LegRoom           INT
Cleanliness       INT
FoodAndDrink      INT
InFlightService   INT
InFlightWifi      INT
InFlightEntertainment INT
BaggageHandling   INT
AvgServiceScore   DECIMAL(5,2)      -- calculé en staging
Satisfaction      NVARCHAR(50)
Satisfaction_Flag BIT               -- calculé en staging

-- stg_flight_activity
LoyaltyNumber           NVARCHAR(50)
Year                    INT
Month                   INT
TotalFlights            INT
Distance                DECIMAL(18,2)
PointsAccumulated       DECIMAL(18,2)
PointsRedeemed          DECIMAL(18,2)
DollarCostPointsRedeemed DECIMAL(18,2)

-- stg_customer_loyalty
LoyaltyNumber      NVARCHAR(50)
Country            NVARCHAR(100)
Province           NVARCHAR(100)
City               NVARCHAR(100)
PostalCode         NVARCHAR(20)
Gender             NVARCHAR(20)
Education          NVARCHAR(100)
Salary             NVARCHAR(50)   -- conservé en staging, non chargé en DWH
MaritalStatus      NVARCHAR(50)
LoyaltyCard        NVARCHAR(50)
CLV                DECIMAL(18,2)  -- converti depuis NVARCHAR raw
EnrollmentType     NVARCHAR(50)
EnrollmentYear     INT
EnrollmentMonth    INT
CancellationYear   INT NULL       -- NULL = membre actif
CancellationMonth  INT NULL

-- stg_date
DateValue       DATE
StartOfYear     DATE
```

```
StartOfQuarter  DATE
StartOfMonth    DATE
```

SQL_Create_DWH_DB

```sql
IF DB_ID(N'DWH_DB') IS NULL
    CREATE DATABASE [DWH_DB];
```

SQL_Create_DWH_Tables

- Crée les 5 tables DIM et 2 tables FACT dans DWH_DB
- `IDENTITY(1,1)` sur toutes les colonnes SK_
- `DEFAULT -1` sur toutes les FK dans les tables FACT
- `QuarterLabel NVARCHAR(20)` (assez large pour 'Unknown' = 7 caractères + 'Q1'..'Q4')

SQL_Unknown_Members

Insère `SK=-1` dans chaque DIM via `SET IDENTITY_INSERT ON/OFF` :

DIM	Valeurs Unknown insérées
`DIM_DATE`	SK=-1, DateValue=NULL, Year=0, Quarter=0, QuarterLabel='Unknown', ...
`DIM_LOYALTY_CARD`	SK=-1, LoyaltyCard='Unknown'
`DIM_GEOGRAPHY`	SK=-1, Country='Unknown', Province='Unknown', City='Unknown', PostalCode='Unknown'
`DIM_TRAVEL`	SK=-1, TypeOfTravel='Unknown', Class='Unknown', TravelProfile='Unknown'
`DIM_CUSTOMER`	SK=-1, toutes les colonnes texte='Unknown', SK_LoyaltyCard=-1, SK_Geography=-1

---

4. Package 01 — `01_Load_Staging.dtsx`

Flux complet

```
SQL_Prepare_Staging
        │
        ├──► SQL_BulkLoad_Satisfaction ──► SQL_Transform_Satisfaction ─┐
        ├──► SQL_BulkLoad_Activity     ──► SQL_Transform_Activity     ─┤
        ├──► SQL_BulkLoad_Customer     ──► SQL_Transform_Customer     ─┤──► SQL_Rebuild_Staging ──►
SQL_Validate_Staging
        └──► SQL_BulkLoad_Date         ──► SQL_Transform_Date         ─┘
             [4 BulkLoads EN PARALLÈLE]    [4 Transforms EN PARALLÈLE]   [LogicalAnd=True sur les 4]
```

SQL_Prepare_Staging

Actions :

1. Supprime les tables `stg_raw_*` résiduelles (IF EXISTS)
2. `TRUNCATE TABLE` sur les 4 tables `stg_*`
3. Supprime les index NC résiduels (IF EXISTS)
4. Crée les 4 tables `stg_raw_*` avec **toutes les colonnes NVARCHAR** :

```sql
CREATE TABLE dbo.stg_raw_satisfaction (
    raw_ID NVARCHAR(20), raw_Gender NVARCHAR(50), raw_Age NVARCHAR(20),
    raw_CustomerType NVARCHAR(100), raw_TypeOfTravel NVARCHAR(100), raw_Class NVARCHAR(100),
    raw_FlightDistance NVARCHAR(20), raw_DepartureDelay NVARCHAR(20), raw_ArrivalDelay NVARCHAR(20),
    raw_TimeConvenience NVARCHAR(20), raw_OnlineBooking NVARCHAR(20), raw_CheckinService NVARCHAR(20),
    raw_OnlineBoarding NVARCHAR(20), raw_GateLocation NVARCHAR(20), raw_OnboardService NVARCHAR(20),
    raw_SeatComfort NVARCHAR(20), raw_LegRoom NVARCHAR(20), raw_Cleanliness NVARCHAR(20),
    raw_FoodAndDrink NVARCHAR(20), raw_InFlightService NVARCHAR(20), raw_InFlightWifi NVARCHAR(20),
    raw_InFlightEntertainment NVARCHAR(20), raw_BaggageHandling NVARCHAR(20), raw_Satisfaction NVARCHAR(50)
);
-- (3 autres tables raw_ similaires)
```

Pourquoi NVARCHAR pour raw_ ? Pour garantir qu'aucune ligne CSV ne soit rejetée par BULK INSERT, même si les données sont mal formatées. La conversion de type se fait ensuite de manière contrôlée avec TRY_CAST.

SQL_Transform_Satisfaction

Source : `stg_raw_satisfaction`
Cible : `stg_airline_satisfaction`
Filtre : `WHERE NULLIF(LTRIM(RTRIM(raw_ID)),'') IS NOT NULL` (élimine les lignes sans ID valide)

Transformations colonne par colonne

Colonne raw_	Transformation SQL appliquée	Colonne stg_	Justification
`raw_ID`	`TRY_CAST(LTRIM(RTRIM(raw_ID)) AS INT)`	`ID` (INT)	Conversion sécurisée — NULL si non numérique
`raw_Gender`	`ISNULL(NULLIF(LTRIM(RTRIM(raw_Gender)),''),'Unknown')`	`Gender` (NVARCHAR)	Trim espaces + NULL-guard
`raw_Age`	`TRY_CAST(LTRIM(RTRIM(raw_Age)) AS INT)`	`Age` (INT)	Conversion sécurisée
`raw_CustomerType`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`CustomerType` (NVARCHAR)	Trim + NULL-guard
`raw_TypeOfTravel`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`TypeOfTravel` (NVARCHAR)	Trim + NULL-guard
`raw_Class`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Class` (NVARCHAR)	Trim + NULL-guard
`raw_FlightDistance`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`FlightDistance` (INT)	Conversion sécurisée
`raw_DepartureDelay`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`DepartureDelay` (**INT NULL**)	NULLIF garde NULL si cellule vide
`raw_ArrivalDelay`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`ArrivalDelay` (**INT NULL**)	~300 NULLs dans la source
`raw_TimeConvenience`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`TimeConvenience` (INT)	Score 0-5
`raw_OnlineBooking`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`OnlineBooking` (INT)	Score 0-5
`raw_CheckinService`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`CheckinService` (INT)	Score 0-5
`raw_OnlineBoarding`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`OnlineBoarding` (INT)	Score 0-5
`raw_GateLocation`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`GateLocation` (INT)	Score 0-5
`raw_OnboardService`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`OnboardService` (INT)	Score 0-5
`raw_SeatComfort`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`SeatComfort` (INT)	Score 0-5
`raw_LegRoom`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`LegRoom` (INT)	Score 0-5
`raw_Cleanliness`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Cleanliness` (INT)	Score 0-5
`raw_FoodAndDrink`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`FoodAndDrink` (INT)	Score 0-5
`raw_InFlightService`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`InFlightService` (INT)	Score 0-5
`raw_InFlightWifi`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`InFlightWifi` (INT)	Score 0-5
`raw_InFlightEntertainment`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`InFlightEntertainment` (INT)	Score 0-5
`raw_BaggageHandling`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`BaggageHandling` (INT)	Score 0-5
(dérivé des 14 scores)	`CAST((score1+score2+…+score14) AS DECIMAL(5,2)) / 14.0`	`AvgServiceScore` (DECIMAL(5,2))	Moyenne arithmétique des 14 dimensions
`raw_Satisfaction`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Satisfaction` (NVARCHAR)	Conservation du texte original

Colonne raw_	Transformation SQL appliquée	Colonne stg_	Justification
(dérivé de Satisfaction)	`CASE WHEN LOWER(LTRIM(RTRIM(raw_Satisfaction)))='satisfied' THEN 1 ELSE 0 END`	`Satisfaction_Flag` (BIT)	Encodage binaire : 1=Satisfait, 0=Insatisfait/Neutre

Colonne `ID` (raw_ID) : chargée en staging mais **non copiée vers DWH** — identifiant technique sans valeur analytique.

SQL_Transform_Activity

Source : `stg_raw_flight_activity`
Cible : `stg_flight_activity`
Filtre : `WHERE NULLIF(LTRIM(RTRIM(raw_LoyaltyNumber)),'') IS NOT NULL`

Colonne raw_	Transformation SQL	Colonne stg_	Type
`raw_LoyaltyNumber`	`LTRIM(RTRIM(...))`	`LoyaltyNumber`	NVARCHAR(50)
`raw_Year`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Year`	INT
`raw_Month`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`Month`	INT
`raw_TotalFlights`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`TotalFlights`	INT
`raw_Distance`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`Distance`	DECIMAL(18,2)
`raw_PointsAccumulated`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`PointsAccumulated`	DECIMAL(18,2)
`raw_PointsRedeemed`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`PointsRedeemed`	DECIMAL(18,2)
`raw_DollarCostPointsRedeemed`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`DollarCostPointsRedeemed`	DECIMAL(18,2)

Note : Les lignes avec `TotalFlights=0` sont **conservées** — un mois sans vol mais avec des points échangés représente une information métier valide.

SQL_Transform_Customer

Source : `stg_raw_customer_loyalty`
Cible : `stg_customer_loyalty`
Filtre : `WHERE NULLIF(LTRIM(RTRIM(raw_LoyaltyNumber)),'') IS NOT NULL`

Colonne raw_	Transformation SQL	Colonne stg_	Type	Note
`raw_LoyaltyNumber`	`LTRIM(RTRIM(...))`	`LoyaltyNumber`	NVARCHAR(50)	Clé naturelle
`raw_Country`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Country`	NVARCHAR(100)	Toujours 'Canada'
`raw_Province`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Province`	NVARCHAR(100)	
`raw_City`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`City`	NVARCHAR(100)	
`raw_PostalCode`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`PostalCode`	NVARCHAR(20)	
`raw_Gender`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Gender`	NVARCHAR(20)	
`raw_Education`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Education`	NVARCHAR(100)	~1 100 NULLs en source
`raw_Salary`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`Salary`	NVARCHAR(50)	**Non chargé en DWH**
`raw_MaritalStatus`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`MaritalStatus`	NVARCHAR(50)	
`raw_LoyaltyCard`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`LoyaltyCard`	NVARCHAR(50)	Star/Nova/Aurora
`raw_CLV`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS DECIMAL(18,2))`	`CLV`	DECIMAL(18,2)	Stocké en VARCHAR dans source
`raw_EnrollmentType`	`ISNULL(NULLIF(LTRIM(RTRIM(...)),''),'Unknown')`	`EnrollmentType`	NVARCHAR(50)	

Colonne raw_	Transformation SQL	Colonne stg_	Type	Note
`raw_EnrollmentYear`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`EnrollmentYear`	INT	
`raw_EnrollmentMonth`	`TRY_CAST(LTRIM(RTRIM(...)) AS INT)`	`EnrollmentMonth`	INT	
`raw_CancellationYear`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`CancellationYear`	**INT NULL**	NULL = membre actif (~80% des lignes)
`raw_CancellationMonth`	`TRY_CAST(NULLIF(LTRIM(RTRIM(...)),'') AS INT)`	`CancellationMonth`	**INT NULL**	NULL si CancellationYear NULL

SQL_Transform_Date

Source : `stg_raw_date`
Cible : `stg_date`
Filtre : `WHERE TRY_CAST(LTRIM(RTRIM(raw_Date)) AS DATE) IS NOT NULL`

Colonne raw_	Transformation SQL	Colonne stg_	Type
`raw_Date`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`DateValue`	DATE
`raw_StartOfYear`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfYear`	DATE
`raw_StartOfQuarter`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfQuarter`	DATE
`raw_StartOfMonth`	`TRY_CAST(LTRIM(RTRIM(...)) AS DATE)`	`StartOfMonth`	DATE

SQL_Rebuild_Staging

Actions post-transform :

1. Supprime les 4 tables `stg_raw_*` (zone raw plus nécessaire)
2. Crée 4 index NC pour accélérer les lookups de la phase DIM :

```
CREATE NONCLUSTERED INDEX IX_stg_sat_id
    ON dbo.stg_airline_satisfaction (ID)
    INCLUDE (TypeOfTravel, Class, Satisfaction_Flag, AvgServiceScore)
    WITH (FILLFACTOR=95);

CREATE NONCLUSTERED INDEX IX_stg_activity_loyalty
    ON dbo.stg_flight_activity (LoyaltyNumber)
    INCLUDE (Year, Month, TotalFlights, Distance, PointsAccumulated)
    WITH (FILLFACTOR=95);

CREATE NONCLUSTERED INDEX IX_stg_customer_loyalty
    ON dbo.stg_customer_loyalty (LoyaltyNumber)
    INCLUDE (LoyaltyCard, Country, Province, City, PostalCode)
    WITH (FILLFACTOR=95);

CREATE NONCLUSTERED INDEX IX_stg_date_value
    ON dbo.stg_date (DateValue)
    INCLUDE (StartOfYear, StartOfQuarter, StartOfMonth)
    WITH (FILLFACTOR=95);
```

3. `UPDATE STATISTICS` sur les 4 tables stg_ avec SAMPLE 50 PERCENT

SQL_Validate_Staging

Génère une erreur bloquante si une table stg_ est vide :

```
IF @sat=0 RAISERROR(N'STAGING VALIDATION FAILED: stg_airline_satisfaction is empty!',16,1);
-- (même chose pour les 3 autres tables)
PRINT CONCAT(N'Staging OK | Satisfaction:',@sat,N' Activity:',@act,N' Customer:',@cust,N' Date:',@dt)
```

---

5. Package 02 — `02_Populate_Dimensions.dtsx`

Règle commune : NOT EXISTS guard

Toutes les insertions DIM utilisent un `WHERE NOT EXISTS` pour être idempotentes (ré-exécutables sans doublons) :

```
WHERE NOT EXISTS (
    SELECT 1 FROM dbo.DIM_xxx d
    WHERE d.[natural_key] = s.[natural_key] AND d.SK_xxx > 0
)
-- SK > 0 exclut la ligne Unknown Member (-1) de la comparaison
```

SQL_DIM_DATE

Source : `STAGING_DB.dbo.stg_date`

Colonne stg_	Transformation / Dérivation SQL	Colonne DIM_DATE
`DateValue`	Copie directe	`DateValue` (DATE)
(dérivé)	`YEAR(DateValue)`	`Year` (INT)
(dérivé)	`DATEPART(QUARTER, DateValue)`	`Quarter` (INT) — 1..4
(dérivé)	`CONCAT(N'Q', DATEPART(QUARTER, DateValue))`	`QuarterLabel` (NVARCHAR(20)) — 'Q1'..'Q4'
(dérivé)	`MONTH(DateValue)`	`MonthNumber` (INT) — 1..12
(dérivé)	`DATENAME(MONTH, DateValue)`	`MonthName` (NVARCHAR(20)) — 'January'..'December'
(dérivé)	`DATEPART(WEEKDAY, DateValue)`	`DayOfWeek` (INT) — 1=Dim..7=Sam
(dérivé)	`DATENAME(WEEKDAY, DateValue)`	`DayName` (NVARCHAR(20)) — 'Sunday'..'Saturday'
(dérivé)	`CASE WHEN DATEPART(WEEKDAY,DateValue) IN (1,7) THEN 1 ELSE 0 END`	`IsWeekend` (BIT)
`StartOfYear`	Copie directe	`StartOfYear` (DATE)
`StartOfQuarter`	Copie directe	`StartOfQuarter` (DATE)
`StartOfMonth`	Copie directe	`StartOfMonth` (DATE)

Résultat : 2 557 lignes (1 ligne = 1 jour du 2012-01-01 au 2018-12-31)

SQL_DIM_LOYALTY_CARD

Source : `STAGING_DB.dbo.stg_customer_loyalty`
Déduplication : `SELECT DISTINCT LoyaltyCard`

Colonne stg_	Transformation	Colonne DIM_LOYALTY_CARD
`LoyaltyCard`	`ISNULL(s.LoyaltyCard, N'Unknown')`	`LoyaltyCard` (NVARCHAR(50))

Résultat : 3 lignes (Star, Nova, Aurora) + SK=-1 Unknown

SQL_DIM_GEOGRAPHY

Source : `STAGING_DB.dbo.stg_customer_loyalty`
Déduplication : `SELECT DISTINCT Country, Province, City, PostalCode`

Colonne stg_	Transformation	Colonne DIM_GEOGRAPHY
`Country`	`ISNULL(s.Country, N'Unknown')`	`Country` (NVARCHAR(100))
`Province`	`ISNULL(s.Province, N'Unknown')`	`Province` (NVARCHAR(100))
`City`	`ISNULL(s.City, N'Unknown')`	`City` (NVARCHAR(100))
`PostalCode`	`ISNULL(s.PostalCode, N'Unknown')`	`PostalCode` (NVARCHAR(20))

Résultat : 55 lignes (55 combinaisons uniques Country+Province+City+PostalCode parmi 16 737 clients)

SQL_DIM_TRAVEL

Source : `STAGING_DB.dbo.stg_airline_satisfaction`
Déduplication : `SELECT DISTINCT TypeOfTravel, Class`

Colonne stg_	Transformation	Colonne DIM_TRAVEL
`TypeOfTravel`	`ISNULL(s.TypeOfTravel, N'Unknown')`	`TypeOfTravel` (NVARCHAR(50))
`Class`	`ISNULL(s.Class, N'Unknown')`	`Class` (NVARCHAR(50))
(dérivé)	`CONCAT(ISNULL(TypeOfTravel,'Unknown'), N' - ', ISNULL(Class,'Unknown'))`	`TravelProfile` (NVARCHAR(100))

Résultat : 6 lignes (Business-Business, Business-Economy, Business-EconomyPlus, Personal-Business, Personal-Economy, Personal-EconomyPlus)

SQL_DIM_CUSTOMER

Prérequis : DIM_LOYALTY_CARD et DIM_GEOGRAPHY doivent être chargées (lookups SK)
Source : `STAGING_DB.dbo.stg_customer_loyalty`
Déduplication : `WHERE NOT EXISTS (...LoyaltyNumber...)`

Colonne stg_	Transformation SQL	Colonne DIM_CUSTOMER	Type
`LoyaltyNumber`	Copie directe	`LoyaltyNumber`	NVARCHAR(50)
`Gender`	`ISNULL(s.Gender, N'Unknown')`	`Gender`	NVARCHAR(20)
`Education`	`ISNULL(s.Education, N'Unknown')`	`Education`	NVARCHAR(100)
`Salary`	`ISNULL(s.Salary, N'Unknown')`	`Salary`	NVARCHAR(50) — présent mais hors analytique
`MaritalStatus`	`ISNULL(s.MaritalStatus, N'Unknown')`	`MaritalStatus`	NVARCHAR(50)
`CLV`	`TRY_CAST(s.CLV AS DECIMAL(18,2))`	`CLV`	DECIMAL(18,2)
`EnrollmentType`	`ISNULL(s.EnrollmentType, N'Unknown')`	`EnrollmentType`	NVARCHAR(50)
`EnrollmentYear`	Copie directe	`EnrollmentYear`	INT
`EnrollmentMonth`	Copie directe	`EnrollmentMonth`	INT
`CancellationYear`	`TRY_CAST(s.CancellationYear AS INT)`	`CancellationYear`	INT NULL
`CancellationMonth`	`TRY_CAST(s.CancellationMonth AS INT)`	`CancellationMonth`	INT NULL
(dérivé)	`CASE WHEN TRY_CAST(CancellationYear AS INT) IS NOT NULL THEN 1 ELSE 0 END`	`IsChurned`	BIT
(lookup)	`ISNULL((SELECT TOP 1 SK_LoyaltyCard FROM DIM_LOYALTY_CARD WHERE LoyaltyCard=ISNULL(s.LoyaltyCard,'Unknown') AND SK_LoyaltyCard>0), -1)`	`SK_LoyaltyCard`	INT
(lookup)	`ISNULL((SELECT TOP 1 SK_Geography FROM DIM_GEOGRAPHY WHERE Country=... AND Province=... AND City=... AND PostalCode=... AND SK_Geography>0), -1)`	`SK_Geography`	INT
(dérivé)	`DATEFROMPARTS(s.EnrollmentYear, s.EnrollmentMonth, 1)`	`EnrollmentDate`	DATE
(init SCD)	`GETDATE()`	`SCD_StartDate`	DATE
(init SCD)	`NULL`	`SCD_EndDate`	DATE NULL
(init SCD)	`1`	`SCD_IsCurrent`	BIT

Résultat : 16 737 lignes

---

6. Package 03 — `03_Populate_Facts.dtsx`

Les deux tâches s'exécutent **en parallèle** (pas de PrecedenceConstraints entre elles).

FACT_PASSENGER_SATISFACTION

Source : `STAGING_DB.dbo.stg_airline_satisfaction`

Colonne FACT	Valeur / Calcul	Note
`SK_Date`	`-1`	Aucune date dans le fichier satisfaction
`SK_Customer`	`-1`	Aucun LoyaltyNumber dans le fichier satisfaction
`SK_Travel`	`ISNULL((SELECT TOP 1 SK_Travel FROM DIM_TRAVEL WHERE TypeOfTravel=s.TypeOfTravel AND Class=s.Class AND SK_Travel>0), -1)`	Lookup sur 2 colonnes
`SK_Geography`	`-1`	Aucune géographie dans le fichier satisfaction
`Gender`	Copie de `s.Gender`	
`CustomerType`	Copie de `s.CustomerType`	
`Age`	Copie de `s.Age`	
`FlightDistance`	Copie de `s.FlightDistance`	
`DepartureDelay`	Copie de `s.DepartureDelay`	INT NULL
`ArrivalDelay`	Copie de `s.ArrivalDelay`	INT NULL
`TimeConvenience` … `BaggageHandling`	Copie directe ×14	Scores 0-5
`AvgServiceScore`	Copie de `s.AvgServiceScore`	Calculé en staging
`Satisfaction`	Copie de `s.Satisfaction`	Texte original
`Satisfaction_Flag`	Copie de `s.Satisfaction_Flag`	Calculé en staging — BIT

Résultat : 129 880 lignes

FACT_FLIGHT_ACTIVITY

Source : `STAGING_DB.dbo.stg_flight_activity`

Colonne FACT	Calcul SQL	Note
`SK_Customer`	`ISNULL((SELECT TOP 1 c.SK_Customer FROM DIM_CUSTOMER c WHERE c.LoyaltyNumber=f.LoyaltyNumber AND c.SK_Customer>0), -1)`	Lookup par LoyaltyNumber
`SK_Date`	`ISNULL((SELECT TOP 1 d.SK_Date FROM DIM_DATE d WHERE d.DateValue=DATEFROMPARTS(f.Year,f.Month,1) AND d.SK_Date>0), -1)`	`DATEFROMPARTS(Year, Month, 1)` → premier du mois
`SK_LoyaltyCard`	`ISNULL((SELECT TOP 1 c.SK_LoyaltyCard FROM DIM_CUSTOMER c WHERE c.LoyaltyNumber=f.LoyaltyNumber AND c.SK_Customer>0), -1)`	Via DIM_CUSTOMER
`SK_Geography`	`ISNULL((SELECT TOP 1 c.SK_Geography FROM DIM_CUSTOMER c WHERE c.LoyaltyNumber=f.LoyaltyNumber AND c.SK_Customer>0), -1)`	Via DIM_CUSTOMER
`TotalFlights`	Copie directe	INT
`Distance`	Copie directe	DECIMAL(18,2)
`PointsAccumulated`	Copie directe	DECIMAL(18,2)
`PointsRedeemed`	Copie directe	DECIMAL(18,2)
`DollarCostPointsRedeemed`	Copie directe	DECIMAL(18,2)

Résultat : 392 936 lignes

---

7. Package 04 — `04_Finalize.dtsx`

Flux

```
[SQL_Indexes_Fact ‖ SQL_Indexes_Dim] → SQL_Update_Stats → SQL_Validate_DWH
```

SQL_Indexes_Fact

```sql
-- Sur FACT_PASSENGER_SATISFACTION
CREATE NONCLUSTERED INDEX IX_fact_sat_travel
    ON dbo.FACT_PASSENGER_SATISFACTION (SK_Travel)
    INCLUDE (Satisfaction_Flag, AvgServiceScore, Gender, CustomerType)
    WITH (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON);

CREATE NONCLUSTERED INDEX IX_fact_sat_flag
    ON dbo.FACT_PASSENGER_SATISFACTION (Satisfaction_Flag)
    INCLUDE (SK_Travel, AvgServiceScore, FlightDistance)
    WITH (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON);

-- Sur FACT_FLIGHT_ACTIVITY
CREATE NONCLUSTERED INDEX IX_fact_activity_customer
    ON dbo.FACT_FLIGHT_ACTIVITY (SK_Customer)
    INCLUDE (SK_Date, TotalFlights, PointsAccumulated)
    WITH (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON);

CREATE NONCLUSTERED INDEX IX_fact_activity_date
    ON dbo.FACT_FLIGHT_ACTIVITY (SK_Date)
    INCLUDE (SK_Customer, TotalFlights, Distance, PointsAccumulated)
    WITH (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON);

CREATE NONCLUSTERED INDEX IX_fact_activity_loyalty
    ON dbo.FACT_FLIGHT_ACTIVITY (SK_LoyaltyCard)
    INCLUDE (TotalFlights, PointsAccumulated, PointsRedeemed)
    WITH (FILLFACTOR=90, DATA_COMPRESSION=PAGE, SORT_IN_TEMPDB=ON);
```

SQL_Indexes_Dim

```sql
CREATE NONCLUSTERED INDEX IX_dim_date_year_month
    ON dbo.DIM_DATE (Year, MonthNumber)
    INCLUDE (Quarter, IsWeekend, QuarterLabel) WITH (FILLFACTOR=95);

CREATE NONCLUSTERED INDEX IX_dim_customer_loyalty
    ON dbo.DIM_CUSTOMER (LoyaltyNumber)
    INCLUDE (IsChurned, SCD_IsCurrent, EnrollmentDate) WITH (FILLFACTOR=90);

CREATE NONCLUSTERED INDEX IX_dim_geo_postalcode
    ON dbo.DIM_GEOGRAPHY (PostalCode)
    INCLUDE (City, Province, Country) WITH (FILLFACTOR=90);

CREATE NONCLUSTERED INDEX IX_dim_travel_profile
    ON dbo.DIM_TRAVEL (TypeOfTravel, Class)
    INCLUDE (TravelProfile) WITH (FILLFACTOR=95);
```

SQL_Update_Stats

```sql
UPDATE STATISTICS dbo.DIM_DATE WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.DIM_LOYALTY_CARD WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.DIM_GEOGRAPHY WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.DIM_CUSTOMER WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.DIM_TRAVEL WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.FACT_PASSENGER_SATISFACTION WITH SAMPLE 50 PERCENT;
UPDATE STATISTICS dbo.FACT_FLIGHT_ACTIVITY WITH SAMPLE 50 PERCENT;
```

SQL_Validate_DWH

Vérifie que chaque table DIM (SK>0) et chaque FACT contient des lignes. En cas d'échec : `RAISERROR(..., 16, 1)` — arrête le package.

---

8. Matrice de lignage complète (CSV → DWH)

airline_passenger_satisfaction.csv → FACT_PASSENGER_SATISFACTION

Colonne CSV	Colonne raw_	Colonne stg_	Traitement	Colonne DWH	Table DWH
ID	raw_ID	ID	TRY_CAST INT	**DROPPED**	—
Gender	raw_Gender	Gender	NULLIF+ISNULL	Gender	FACT_PASSENGER_SATISFACTION
Age	raw_Age	Age	TRY_CAST INT	Age	FACT_PASSENGER_SATISFACTION
Customer Type	raw_CustomerType	CustomerType	NULLIF+ISNULL	CustomerType	FACT_PASSENGER_SATISFACTION
Type of Travel	raw_TypeOfTravel	TypeOfTravel	NULLIF+ISNULL	→ SK_Travel (via DIM_TRAVEL)	FACT_PASSENGER_SATISFACTION
Class	raw_Class	Class	NULLIF+ISNULL	→ SK_Travel (via DIM_TRAVEL)	FACT_PASSENGER_SATISFACTION
Flight Distance	raw_FlightDistance	FlightDistance	TRY_CAST INT	FlightDistance	FACT_PASSENGER_SATISFACTION
Departure Delay	raw_DepartureDelay	DepartureDelay	TRY_CAST NULLIF INT NULL	DepartureDelay	FACT_PASSENGER_SATISFACTION
Arrival Delay	raw_ArrivalDelay	ArrivalDelay	TRY_CAST NULLIF INT NULL	ArrivalDelay	FACT_PASSENGER_SATISFACTION
Dep/Arr Time Convenience	raw_TimeConvenience	TimeConvenience	TRY_CAST INT	TimeConvenience	FACT_PASSENGER_SATISFACTION
Ease of Online Booking	raw_OnlineBooking	OnlineBooking	TRY_CAST INT	OnlineBooking	FACT_PASSENGER_SATISFACTION
Check-in Service	raw_CheckinService	CheckinService	TRY_CAST INT	CheckinService	FACT_PASSENGER_SATISFACTION
Online Boarding	raw_OnlineBoarding	OnlineBoarding	TRY_CAST INT	OnlineBoarding	FACT_PASSENGER_SATISFACTION
Gate Location	raw_GateLocation	GateLocation	TRY_CAST INT	GateLocation	FACT_PASSENGER_SATISFACTION
On-board Service	raw_OnboardService	OnboardService	TRY_CAST INT	OnboardService	FACT_PASSENGER_SATISFACTION
Seat Comfort	raw_SeatComfort	SeatComfort	TRY_CAST INT	SeatComfort	FACT_PASSENGER_SATISFACTION
Leg Room Service	raw_LegRoom	LegRoom	TRY_CAST INT	LegRoom	FACT_PASSENGER_SATISFACTION
Cleanliness	raw_Cleanliness	Cleanliness	TRY_CAST INT	Cleanliness	FACT_PASSENGER_SATISFACTION
Food and Drink	raw_FoodAndDrink	FoodAndDrink	TRY_CAST INT	FoodAndDrink	FACT_PASSENGER_SATISFACTION
In-flight Service	raw_InFlightService	InFlightService	TRY_CAST INT	InFlightService	FACT_PASSENGER_SATISFACTION
In-flight Wifi Service	raw_InFlightWifi	InFlightWifi	TRY_CAST INT	InFlightWifi	FACT_PASSENGER_SATISFACTION
In-flight Entertainment	raw_InFlightEntertainment	InFlightEntertainment	TRY_CAST INT	InFlightEntertainment	FACT_PASSENGER_SATISFACTION
Baggage Handling	raw_BaggageHandling	BaggageHandling	TRY_CAST INT	BaggageHandling	FACT_PASSENGER_SATISFACTION
Satisfaction	raw_Satisfaction	Satisfaction	NULLIF+ISNULL	Satisfaction	FACT_PASSENGER_SATISFACTION
(dérivé)	—	AvgServiceScore	(14 cols)/14.0	AvgServiceScore	FACT_PASSENGER_SATISFACTION
(dérivé)	—	Satisfaction_Flag	CASE LOWER='satisfied' THEN 1 ELSE 0	Satisfaction_Flag	FACT_PASSENGER_SATISFACTION

Customer Loyalty History.csv → DIM_CUSTOMER + DIM_GEOGRAPHY + DIM_LOYALTY_CARD

Colonne CSV	Colonne stg_	Traitement	Colonne DWH	Table DWH
Loyalty Number	LoyaltyNumber	LTRIM/RTRIM	LoyaltyNumber (NK)	DIM_CUSTOMER
Country	Country	NULLIF+ISNULL	Country	DIM_GEOGRAPHY
Province	Province	NULLIF+ISNULL	Province	DIM_GEOGRAPHY
City	City	NULLIF+ISNULL	City	DIM_GEOGRAPHY
Postal Code	PostalCode	NULLIF+ISNULL	PostalCode	DIM_GEOGRAPHY
Gender	Gender	NULLIF+ISNULL	Gender	DIM_CUSTOMER
Education	Education	NULLIF+ISNULL	Education	DIM_CUSTOMER
Salary	Salary	NULLIF+ISNULL	**DROPPED**	—
Marital Status	MaritalStatus	NULLIF+ISNULL	MaritalStatus	DIM_CUSTOMER
Loyalty Card	LoyaltyCard	NULLIF+ISNULL	LoyaltyCard	DIM_LOYALTY_CARD
CLV	CLV	TRY_CAST DECIMAL	CLV	DIM_CUSTOMER
Enrollment Type	EnrollmentType	NULLIF+ISNULL	EnrollmentType	DIM_CUSTOMER
Enrollment Year	EnrollmentYear	TRY_CAST INT	EnrollmentYear	DIM_CUSTOMER
Enrollment Month	EnrollmentMonth	TRY_CAST INT	EnrollmentMonth	DIM_CUSTOMER
Cancellation Year	CancellationYear	TRY_CAST NULLIF INT NULL	CancellationYear	DIM_CUSTOMER
Cancellation Month	CancellationMonth	TRY_CAST NULLIF INT NULL	CancellationMonth	DIM_CUSTOMER
(dérivé)	—	CancellationYear IS NOT NULL → 1 ELSE 0	IsChurned	DIM_CUSTOMER
(dérivé)	—	DATEFROMPARTS(EYear, EMonth, 1)	EnrollmentDate	DIM_CUSTOMER
(lookup)	—	SELECT SK_LoyaltyCard WHERE LoyaltyCard=s.LoyaltyCard	SK_LoyaltyCard	DIM_CUSTOMER
(lookup)	—	SELECT SK_Geography WHERE Country+Province+City+PostalCode match	SK_Geography	DIM_CUSTOMER
(init SCD)	—	GETDATE() / NULL / 1	SCD_StartDate / SCD_EndDate / SCD_IsCurrent	DIM_CUSTOMER

Customer Flight Activity.csv → FACT_FLIGHT_ACTIVITY

Colonne CSV	Colonne stg_	Traitement	Colonne DWH	Table DWH
Loyalty Number	LoyaltyNumber	LTRIM/RTRIM	→ SK_Customer (via DIM_CUSTOMER)	FACT_FLIGHT_ACTIVITY
Year	Year	TRY_CAST INT	→ SK_Date (DATEFROMPARTS)	FACT_FLIGHT_ACTIVITY
Month	Month	TRY_CAST INT	→ SK_Date (DATEFROMPARTS)	FACT_FLIGHT_ACTIVITY
Total Flights	TotalFlights	TRY_CAST INT	TotalFlights	FACT_FLIGHT_ACTIVITY
Distance	Distance	TRY_CAST DECIMAL	Distance	FACT_FLIGHT_ACTIVITY
Points Accumulated	PointsAccumulated	TRY_CAST DECIMAL	PointsAccumulated	FACT_FLIGHT_ACTIVITY
Points Redeemed	PointsRedeemed	TRY_CAST DECIMAL	PointsRedeemed	FACT_FLIGHT_ACTIVITY

Colonne CSV	Colonne stg_	Traitement	Colonne DWH	Table DWH
Dollar Cost Points Redeemed	DollarCostPointsRedeemed	TRY_CAST DECIMAL	DollarCostPointsRedeemed	FACT_FLIGHT_ACTIVITY

Calendar.csv → DIM_DATE

Colonne CSV	Colonne stg_	Traitement	Colonne DWH
Date	DateValue	TRY_CAST DATE	DateValue
Start of Year	StartOfYear	TRY_CAST DATE	StartOfYear
Start of Quarter	StartOfQuarter	TRY_CAST DATE	StartOfQuarter
Start of Month	StartOfMonth	TRY_CAST DATE	StartOfMonth
(dérivé)	—	YEAR(DateValue)	Year
(dérivé)	—	DATEPART(QUARTER, DateValue)	Quarter
(dérivé)	—	CONCAT('Q', DATEPART(QUARTER,...))	QuarterLabel
(dérivé)	—	MONTH(DateValue)	MonthNumber
(dérivé)	—	DATENAME(MONTH, DateValue)	MonthName
(dérivé)	—	DATEPART(WEEKDAY, DateValue)	DayOfWeek
(dérivé)	—	DATENAME(WEEKDAY, DateValue)	DayName
(dérivé)	—	CASE WHEN WEEKDAY IN (1,7) THEN 1 ELSE 0 END	IsWeekend

---

9. Résumé des résultats de chargement

Table	Lignes chargées	Source
DIM_DATE	2 557	Calendar.csv
DIM_LOYALTY_CARD	3	Customer Loyalty History.csv
DIM_GEOGRAPHY	55	Customer Loyalty History.csv
DIM_CUSTOMER	16 737	Customer Loyalty History.csv
DIM_TRAVEL	6	airline_passenger_satisfaction.csv
FACT_PASSENGER_SATISFACTION	129 880	airline_passenger_satisfaction.csv
FACT_FLIGHT_ACTIVITY	392 936	Customer Flight Activity.csv
TOTAL DWH	**542 139**	

---

10. Qualité des données — Synthèse

#	Problème source	Action appliquée	Outil	Résultat
1	Cellules vides dans CSV → chaîne vide en NVARCHAR	`NULLIF(LTRIM(RTRIM(...)),'')`	T-SQL inline	NULL propre ou 'Unknown'
2	`ArrivalDelay` / `DepartureDelay` parfois NULL	`TRY_CAST(NULLIF(...,'') AS INT)` → INT NULL	T-SQL inline	NULL conservé (pas d'imputation)
3	`CLV` stocké en VARCHAR dans source	`TRY_CAST(NULLIF(...,'') AS DECIMAL(18,2))`	T-SQL inline	DECIMAL propre
4	`CancellationYear/Month` vides = membres actifs	`TRY_CAST(NULLIF(...,'') AS INT)` → NULL	T-SQL inline	NULL = actif, INT = churné
5	`Satisfaction` texte mixte → besoin binaire	`CASE WHEN LOWER(...)='satisfied' THEN 1 ELSE 0`	T-SQL inline	BIT propre (1/0)
6	14 scores de service → besoin agrégat	`(sum14) / 14.0` → DECIMAL(5,2)	T-SQL inline	AvgServiceScore pré-calculé
7	`TypeOfTravel` + `Class` → clé DIM_TRAVEL	`CONCAT(... ' - ' ...)` → TravelProfile	T-SQL inline	6 combinaisons distinctes

#	Problème source	Action appliquée	Outil	Résultat
8	`Salary` — valeurs négatives, peu fiable	Non chargé dans DWH (conservé en staging)	Exclusion	Hors périmètre analytique
9	Score `0` = service non applicable	Conservé tel quel	Documenté	Exclu via `WHERE score > 0` dans les requêtes
10	UTF-8 BOM sur fichiers Loyalty et Activity	`CODEPAGE='65001'` dans BULK INSERT	BULK INSERT	BOM géré nativement
11	`EnrollmentYear/Month` séparés → date	`DATEFROMPARTS(year, month, 1)`	T-SQL inline	`EnrollmentDate` DATE = YYYY-MM-01
12	Doublons potentiels lors de rechargements	`WHERE NOT EXISTS` sur toutes les insertions DIM	T-SQL inline	Idempotence garantie

---

Airline Analytics DWH — Esprit Engineering, 3ème année — 2026-04-24